PORTFOLIO PROJECT BLUEPRINT

ShopPulse

A complete data engineering platform for real-time e-commerce analytics

This guide explains the purpose, architecture, data model, pipeline, metrics, quality controls, and build plan in beginner-friendly language.

CORE STACK	OUTCOME
Docker | PostgreSQL | Kafka | Kestra | GCS | Spark | BigQuery | dbt | Terraform	A working pipeline that turns store activity into trustworthy dashboards and portfolio evidence.

Prepared as an implementation-ready learning guide

ORIENTATION

How to use this guide

Read pages 3-5 first to understand the business story and end-to-end flow. Then use the remaining pages as a build reference.

03 Project purpose
The problem, users, and definition of value

04 System architecture
How every technology fits together

05 One customer's journey
A concrete event moving through the pipeline

06 Source systems and data
Tables, events, ownership, and grain

07 Streaming with Kafka
Topics, messages, consumers, and guarantees

08 Data lake and Spark
Raw history, cleaning, partitioning, sessions

09 BigQuery and dbt
Warehouse layers, facts, dimensions, and tests

10 Metrics and dashboard
Revenue, funnels, products, and payments

11 Orchestration and reliability
Schedules, retries, quality gates, observability

12 Infrastructure and cost
Docker, Terraform, security, and cost control

13 Build roadmap
Five phases from MVP to portfolio-ready

14 Repository and workflow
Folders, commands, environments, and CI

15 Demo and definition of done
How to prove the project works

16 Glossary and decisions
Key terms and recommended defaults

The most important design rule

Do not add a tool only because it appears in a course. Every component must solve a specific problem, and the first version should work end to end before streaming and scale are added.

01 / PURPOSE

Why ShopPulse should exist

ShopPulse gives an online store a single, trustworthy view of sales, customer behavior, product performance, and payment health.

Business purpose

Help a store manager answer practical questions: What did we sell? Which products perform best? Where do shoppers leave? Are payment failures increasing?

Learning purpose

Prove that you can design, build, test, explain, and operate a modern data platform from ingestion to a business-facing dashboard.

The situation

An online store produces two different kinds of information. Its operational database records customers, products, orders, and payments. Its website produces a much faster stream of views, searches, cart actions, and checkout attempts.

Those records are useful individually, but business decisions require them to be combined, cleaned, and interpreted consistently. ShopPulse performs that work while keeping the original history available for recovery and reprocessing.

Primary users

Store manager	A quick view of business performance	Revenue, orders, customers, conversion, trends
Marketing analyst	Understand acquisition and behavior	Sessions, channels, product views, funnel drop-off
Operations team	Detect payment or order problems	Failure rates, cancellations, freshness and alerts
Data engineer	A reliable and explainable pipeline	Schedules, logs, tests, lineage, replayable raw data

Success statement

A manager can open one dashboard and trust that its metrics are fresh, traceable to source records, protected by tests, and reproducible from raw data.

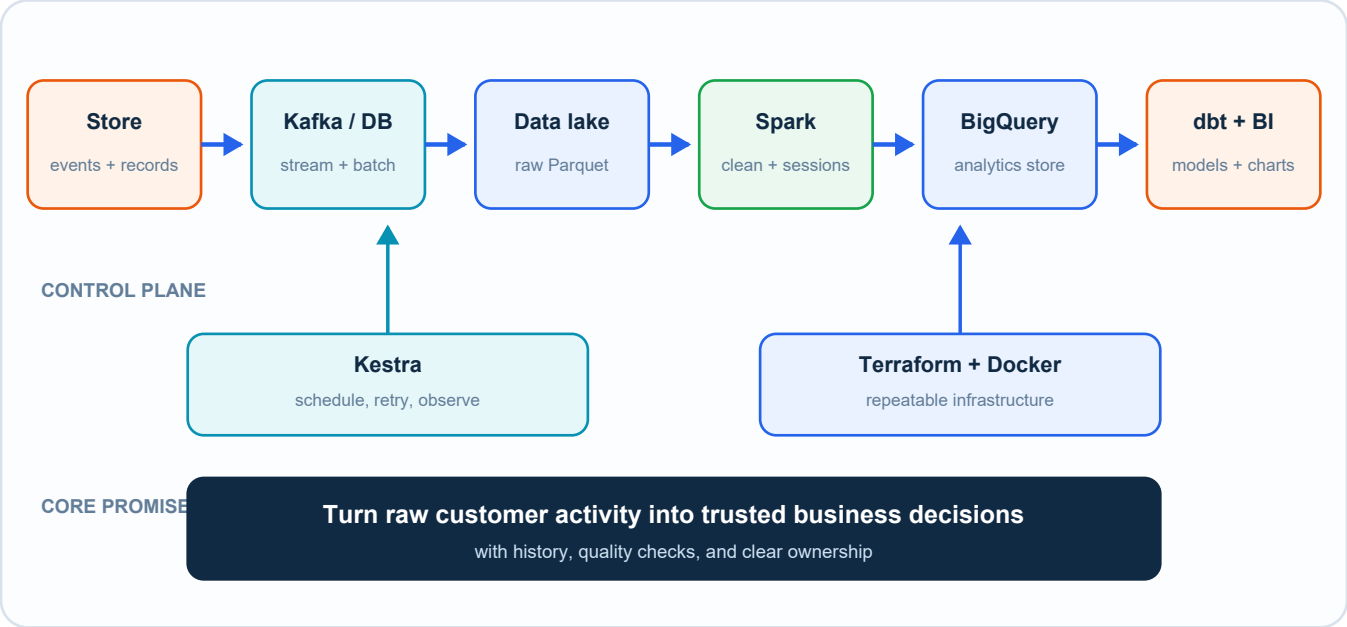
Questions the finished system must answer

- How much revenue did we make today, this week, and this month?
- Which products are viewed often but purchased rarely?
- What percentage of sessions reach cart, checkout, and purchase?
- Which payment methods have the highest failure rate?
- How long does data take to appear on the dashboard?

02 / ARCHITECTURE

The complete project pipeline

The platform has a data plane that moves information and a control plane that schedules, configures, and observes the work.



PostgreSQL	Operational source	Stores customers, products, orders, and payments
Kafka	Streaming transport	Moves live behavior events without tightly coupling producers and consumers
GCS data lake	Raw historical storage	Keeps inexpensive, replayable, partitioned files
Spark	Large event processing	Deduplicates, validates, joins, and sessionizes event data
BigQuery	Analytics warehouse	Answers large SQL queries and serves transformed models
dbt	Business transformations	Creates documented, tested staging, fact, dimension, and mart models
Kestra	Workflow orchestration	Schedules jobs, passes parameters, retries failures, and records runs
Docker	Local reproducibility	Starts the same development services on any machine
Terraform	Cloud reproducibility	Creates buckets, datasets, and permissions from reviewed code

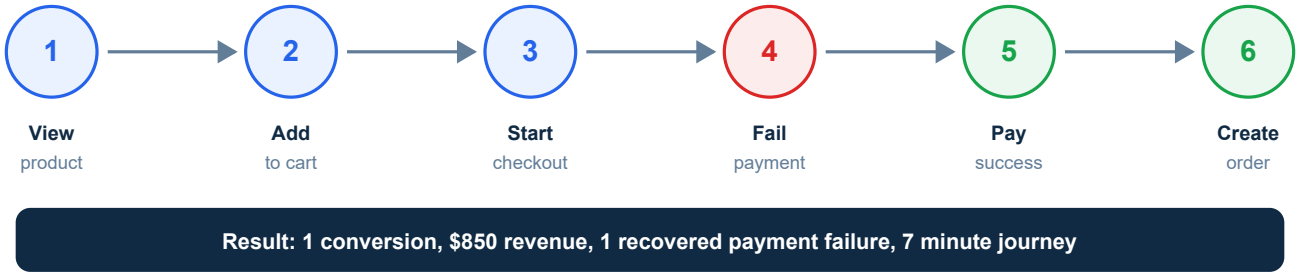
Freshness targets

Streaming behavior metrics should arrive within 2-5 minutes. Orders and reference tables may refresh hourly in the MVP. Daily aggregates should be complete by 06:00 local time.

03 / CONCRETE EXAMPLE

Follow one customer through the platform

A concrete journey makes the architecture easier to understand than a list of tools.



Result: 1 conversion, $850 revenue, 1 recovered payment failure, 7 minute journey

1. Ada views a laptop

The website publishes a product_viewed event to Kafka with a unique event ID, session ID, product ID, timestamp, channel, and device.

2. She adds it to her cart

A cart event is published. The event consumer copies both messages into the raw data lake without changing their business meaning.

3. She starts checkout

The checkout_started event lets the funnel model distinguish buying intent from a simple product view.

4. Her first payment fails

The store records the failed attempt in PostgreSQL and publishes a payment result event. Operations can later measure failure rates by method.

5. Her second payment succeeds

A successful payment and completed order are stored. The order is the official revenue source, while behavior events explain the journey.

6. The pipeline builds the answer

Spark groups events into one session. BigQuery receives clean records. dbt joins the session, order, payment, customer, and product models.

What the dashboard finally sees

One completed order, $850 revenue, one recovered payment failure, three product views, one cart addition, one checkout, and approximately seven minutes from first view to purchase.

Example event contract

```
{
  "event_id": "evt_10291",
  "event_type": "product_viewed",
  "customer_id": "cust_421",
  "session_id": "session_702",
  "product_id": "product_91",
  "event_time": "2026-09-07T14:30:00Z",
  "device": "mobile"
}
```

04 / SOURCES

What data enters the system

The project intentionally separates official business records from high-volume behavior events.

System of record: PostgreSQL

The final truth for customers, products, orders, order items, payments, and refunds. Updates are extracted incrementally using timestamps or monotonically increasing IDs.

Behavior stream: Kafka

The chronological story of sessions, searches, views, carts, checkouts, and payment outcomes. Events are append-only and carry stable identifiers.

Operational tables

customers	One row per customer	customer_id, country, signup_at, acquisition_channel
products	One row per product	product_id, category_id, name, price, active
orders	One row per order	order_id, customer_id, ordered_at, status, total_amount
order_items	One row per product in an order	order_id, product_id, quantity, unit_price
payments	One row per payment attempt	payment_id, order_id, method, status, amount, attempted_at
refunds	One row per refund	refund_id, payment_id, amount, reason, refunded_at

Required fields on every behavior event

event_id	Idempotency and duplicate removal
event_type	Routing and metric logic
event_time	When the action happened at the source
ingested_at	When the platform received the event
session_id	Groups anonymous or authenticated actions into a visit
customer_id	Nullable until the visitor signs in or checks out
schema_version	Allows safe evolution of the message contract

Data ownership rule

Orders and revenue come from the operational database. Kafka explains behavior and speed, but it must not invent financial truth. This prevents double counting when an event is retried.

05 / STREAMING

Kafka design and event guarantees

Kafka lets the store publish customer activity once while independent consumers store, monitor, or process it at their own pace.

shop.web_events	session_started, searched, product_viewed	session_id	7 days
shop.cart_events	added_to_cart, removed_from_cart	session_id	7 days
shop.checkout_events	checkout_started, order_submitted	session_id	14 days
shop.payment_events	payment_failed, payment_completed	order_id	30 days
shop.dead_letter	Messages that cannot be validated	original topic	30 days

Message lifecycle

1. The producer creates an event ID before publishing and validates required fields against the event contract.
2. Kafka assigns the message to a partition. A stable partition key preserves order for one session or order.
3. A raw sink consumer writes the event to date- and hour-partitioned Parquet files in the lake.
4. The consumer commits its Kafka offset only after storage succeeds. On restart, an event may be read again.
5. Spark removes duplicates using event_id and keeps the latest valid ingestion record.
6. Invalid messages go to the dead-letter topic with an error reason for later inspection.

Delivery choice: at least once

A message might be delivered more than once, but it should not disappear silently. Downstream deduplication makes repeated delivery safe and easier to operate.

Late events

Event time is different from ingestion time. Allow a lateness window, update recent partitions, and record how many events arrive after that window.

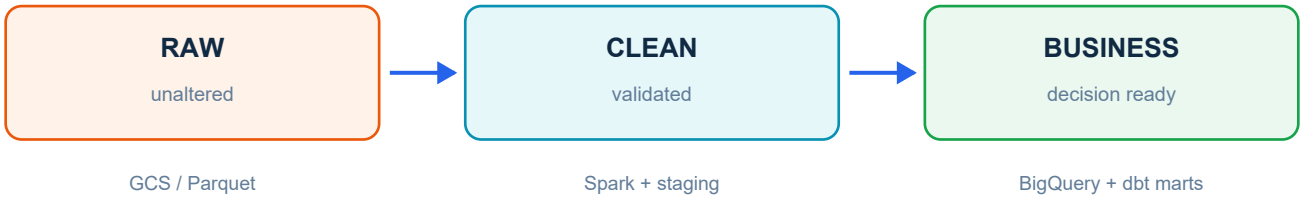
MVP simplification

Build the first working pipeline without Kafka. Then add streaming as Phase 3. This keeps early debugging focused and ensures the dashboard already has a trusted batch foundation.

06 / STORAGE AND PROCESSING

Data lake layout and Spark jobs

The lake preserves history; Spark turns messy event files into validated, query-friendly datasets.



Recommended object layout

```
gs://shoppulse-data/
  raw/postgres/orders/load_date=2026-09-07/
  raw/kafka/web_events/event_date=2026-09-07/hour=14/
  quarantine/events/error_date=2026-09-07/
  clean/events/event_date=2026-09-07/
  clean/sessions/session_date=2026-09-07/
```

Spark responsibilities

clean_events	Raw Kafka files	clean/events	Schema checks, timestamp normalization, duplicate removal
build_sessions	Clean events	clean/sessions	Group by session, calculate duration and funnel milestones
enrich_events	Clean events + products	clean/enriched_events	Attach category, current price, and product attributes
late_event_repair	Recent raw partitions	Updated clean partitions	Reprocess a rolling window to capture late arrivals

Partitioning rule

Partition on fields commonly used to limit scans, usually event_date or order_date. Avoid very high-cardinality partitions such as customer_id, session_id, or event_id.

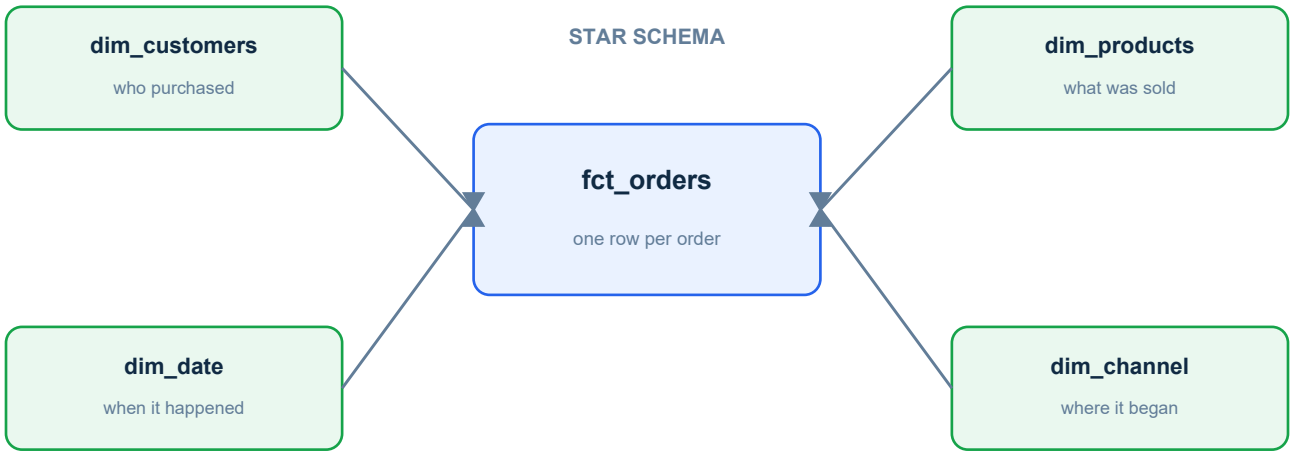
Raw-zone promises

- Never overwrite original source meaning; add ingestion metadata instead.
- Use stable file formats and retain enough history to replay a failed transformation.
- Record source, extraction time, row count, file count, and pipeline run ID.
- Quarantine malformed data rather than mixing it into valid business datasets.

07 / ANALYTICS ENGINEERING

BigQuery warehouse and dbt models

BigQuery provides scalable SQL storage; dbt turns raw tables into tested models with business meaning.



Source	raw_*	Data as loaded from lake or database	raw_orders, raw_events
Staging	stg_*	Rename, cast, deduplicate, standardize	stg_orders, stg_payments
Intermediate	int_*	Reusable joins and business logic	int_order_payments, int_sessions
Core	fct_* / dim_*	Stable facts and dimensions at declared grain	fct_orders, dim_products
Mart	mart_*	Dashboard-ready aggregates	mart_daily_sales, mart_funnel

Tests are part of the product

unique + not_null	Two rows claiming to be the same order
relationships	An order referencing a customer that does not exist
accepted_values	An unknown payment status entering metrics
business rule	A completed order with no successful payment
freshness	A dashboard silently showing yesterday's data
reconciliation	Warehouse revenue not matching the operational source

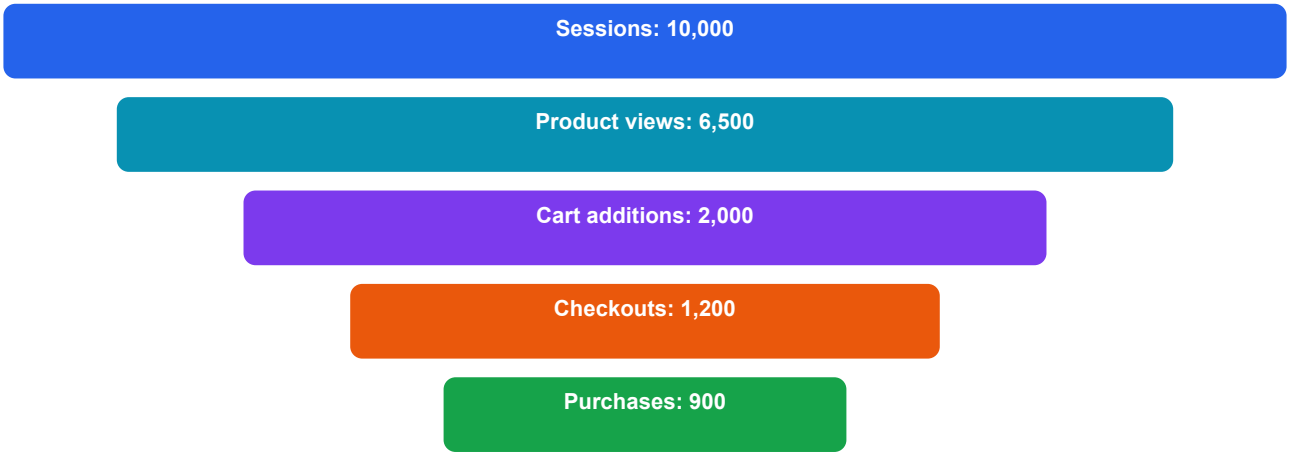
Declare the grain

Every fact model begins with one sentence: one row represents exactly one ____. For fct_orders it is one order; for fct_payments it is one payment attempt; for fct_sessions it is one website session.

08 / BUSINESS OUTPUT

Metrics and dashboard design

The dashboard is where pipeline work becomes visible. Every chart must have a clear owner, formula, refresh expectation, and business action.



Overall conversion = 900 / 10,000 = 9%

Revenue	Sum of completed, non-refunded order value	Measures realized sales
Orders	Count of completed order IDs	Measures transaction volume
Average order value	Revenue / completed orders	Tracks basket value
Conversion rate	Purchasing sessions / all eligible sessions	Measures funnel effectiveness
Cart abandonment	Cart sessions without purchase / cart sessions	Shows lost buying intent
Payment success rate	Successful attempts / all final attempts	Monitors checkout reliability
Repeat customer rate	Customers with 2+ completed orders / purchasing customers	Measures retention
Data freshness	Current time - latest successful source event	Shows whether metrics are current

Recommended dashboard pages

Executive overview	Revenue, orders, customers, AOV, trends, geography
Product performance	Top products, category trends, view-to-purchase rate, low conversion
Customer funnel	Sessions, views, carts, checkouts, purchases, drop-off by device/channel
Payment health	Success rate, failures by method, recovered failures, affected revenue
Data health	Freshness, row counts, failed tests, late events, last successful pipeline run

Metric contract example

Revenue uses order currency at transaction time, excludes cancelled orders, subtracts completed refunds, uses the store's reporting timezone, and refreshes hourly. Write rules like this before building charts.

09 / OPERATIONS

Kestra workflows and reliability controls

Orchestration is more than scheduling: it coordinates dependencies, records state, retries safely, and stops bad data from reaching users.

hourly_orders	Hourly	Extract changed records -> lake -> BigQuery	Watermark advanced and counts reconciled
stream_to_lake	Continuous	Consume -> validate -> write -> commit	Offsets progress and error rate stays below threshold
event_processing	Every 15 min	Spark clean -> sessionize -> load	Recent event partitions complete
dbt_build	After loads	Sources -> staging -> core -> marts -> tests	All blocking tests pass
daily_quality	Daily 06:00	Freshness -> volume -> revenue reconciliation	No critical anomalies

Safe workflow behavior

Idempotency

Running the same interval twice produces the same final result. Use stable IDs, merge logic, and processed watermarks.

Retries

Retry transient network or service errors with delay. Do not endlessly retry invalid data or broken business logic.

Backfills

Accept start_date and end_date parameters so historical periods can be rebuilt deliberately.

Quality gates

Stop publication when critical dbt tests, revenue reconciliation, or freshness checks fail.

Observability

Record run ID, start and end time, source watermark, rows read/written/rejected, and error details.

Warning

A non-critical volume shift or a small number of late events. Continue the pipeline, but make the condition visible.

Failure

Missing required fields, broken relationships, stale source data, or unreconciled financial totals. Stop publishing affected marts.

Why Bruin is optional

Bruin overlaps with parts of Kestra and dbt. Keep it outside the MVP. Evaluate it later in a separate branch or comparison so the main architecture stays easy to explain.

10 / PLATFORM FOUNDATION

Infrastructure, security, and cost control

The project should be easy to reproduce without treating cloud credentials or uncontrolled spending casually.

Docker Compose	PostgreSQL, Kafka, Kestra, event generator	Not used to create managed cloud services
Terraform	Optional local variables and validation	GCS bucket, BigQuery datasets, service accounts, permissions
Environment files	Non-secret configuration	Project ID, region, dataset and bucket names
Secret store	Developer credentials outside Git	Service account or workload identity secrets

Security minimums

- Never commit passwords, private keys, connection strings, or cloud credential files.
- Give each service only the permissions it needs; separate read, write, and administration roles.
- Use generated names and fictional customer data. Do not place real personal information in the portfolio dataset.
- Encrypt traffic and storage using managed defaults, then document what protects each boundary.
- Mask or omit names, emails, and addresses from analytics models unless a use case genuinely requires them.

Cost-control defaults

BigQuery	Partition facts by date, cluster on common filters, require partition filters, set query budgets
GCS	Compress as Parquet, lifecycle old demo files, keep raw retention deliberate
Spark	Use local mode during development; run small cloud jobs only when demonstrating scale
Kafka	Run locally in Docker for the portfolio; avoid paying for a managed cluster
Data volume	Use enough rows to demonstrate behavior, not enough to create an unnecessary bill

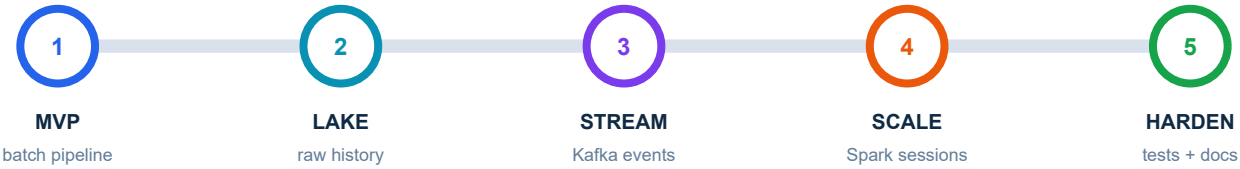
Suggested portfolio scale

5,000 customers, 200 products, 20,000 sessions, 2,000 orders, and a controllable live event stream. This is large enough for realistic metrics and small enough to run cheaply.

11 / DELIVERY PLAN

Build in five controlled phases

Each phase ends with something demonstrable. Do not begin the next phase until the current path is working and tested.



1 - MVP	Generator -> PostgreSQL -> Kestra -> BigQuery -> dbt -> one dashboard	One command starts local services; one workflow produces trusted daily sales
2 - Data lake	Write incremental source extracts to partitioned GCS Parquet before BigQuery	Delete a derived table and rebuild it from raw history
3 - Streaming	Publish website events to Kafka; sink them into the raw lake	Live product-view and cart counts appear within five minutes
4 - Spark	Clean, deduplicate, enrich, and sessionize event files	Late and duplicate events do not corrupt funnel metrics
5 - Harden	Terraform, CI, tests, alerts, lineage, docs, demo script	A new reviewer can deploy, run, understand, and verify the project

Suggested order inside Phase 1

1. Define metrics and source schemas before writing pipeline code.
2. Generate customers, products, orders, order items, and payment attempts.
3. Start PostgreSQL and load a deterministic seed dataset.
4. Create an incremental Kestra extraction using updated_at watermarks.
5. Load BigQuery raw tables and build dbt staging, facts, dimensions, and daily sales.
6. Add tests, then connect a minimal dashboard to the marts.
7. Document how to run the demonstration from a clean machine.

Scope guard

If Phase 1 is not complete, do not add Spark, Kubernetes, machine learning, a second cloud, or a complex frontend. A small finished system is stronger than a large collection of half-connected tools.

12 / REPOSITORY

Recommended folder structure and workflow

The repository should communicate ownership: infrastructure, ingestion, processing, transformations, quality, and documentation each have an obvious home.

```
shoppulse/
  README.md
  docker-compose.yml
  .env.example
  data-generator/
  ingestion/postgres/
  ingestion/kafka/
  orchestration/kestra/
  processing/spark/
  warehouse/dbt/
  infrastructure/terraform/
  dashboards/
  tests/
  docs/architecture/
```

Local	Fast development	Docker services, seed data, local Kafka and Spark, safe cloud sandbox
CI	Automated verification	Formatting, unit tests, dbt compile/tests, Terraform validation
Demo	Portfolio presentation	Small deterministic dataset plus controllable streaming events

README checklist

- Problem statement, intended users, and screenshots of the final dashboard.
- Architecture diagram plus a short explanation of every component and tradeoff.
- Prerequisites, setup, teardown, configuration, and expected runtime/cost.
- Data dictionary, model grain, metric definitions, and dbt documentation link or export.
- How to run a backfill, recover a failed job, and inspect quarantined events.
- Known limitations and a deliberately small list of future improvements.

Branch discipline

Build one vertical capability at a time. Keep changes reviewable and pair code changes with tests and documentation.

Reproducibility

Use fixed random seeds for demo data, pinned dependency versions, and infrastructure code for cloud resources.

Environment contract

A reviewer should be able to copy .env.example, add their own safe credentials, start services, generate data, run the pipeline, and see the expected metrics without editing source code.

13 / PROOF

Demo story and definition of done

The project is complete when you can prove correct behavior, explain tradeoffs, and recover from ordinary failures.

A strong 8-minute portfolio demonstration

1. State the problem: store information is split between transactions and fast behavior events.
2. Show the architecture and explain why each component exists in one sentence.
3. Start a small event stream and show Kafka receiving views, carts, and checkout events.
4. Open Kestra and show the pipeline run, parameters, row counts, and successful quality gate.
5. Query one clean BigQuery fact model and show its declared grain.
6. Open dbt documentation and point to tests and lineage for mart_daily_sales.
7. Show the dashboard changing, then explain one business insight from the funnel.
8. Demonstrate recovery: replay a small interval or show a malformed message in quarantine.

Definition of done

Correctness	Key metrics reconcile to known source totals and critical dbt tests pass
Freshness	The dashboard exposes last update time and meets documented targets
Reliability	Retries are safe, duplicate events are harmless, and backfills are parameterized
Recoverability	Derived data can be rebuilt from retained raw history
Security	No secrets or real personal data exist in Git or generated artifacts
Cost	Cloud resources, partition rules, cleanup, and expected demo cost are documented
Usability	A reviewer can run the project from the README without private knowledge
Communication	Architecture, data model, metric rules, limitations, and tradeoffs are explained

Portfolio test

If you can explain how one product_viewed message becomes a funnel metric, how duplicate delivery is handled, and why revenue comes from orders rather than behavior events, you understand the heart of the project.

14 / REFERENCE

Glossary and recommended decisions

Use this page when unfamiliar terms appear during implementation.

Batch	Process a collected group of records on a schedule
Stream	Process a continuing flow of events shortly after they happen
Event	An immutable statement that something happened at a specific time
Grain	Exactly what one row represents
Watermark	The latest source position or timestamp successfully processed
Idempotent	Safe to run more than once without changing the correct final result
Partition	A physical or logical slice of data used to limit work and cost
Fact	A measurable business occurrence such as an order or payment attempt
Dimension	Descriptive context such as customer, product, date, or channel
Data mart	A model shaped for a specific business question or dashboard
Lineage	A map showing which sources and transformations produced a model
Backfill	Reprocess a historical time range deliberately
Dead-letter topic	A holding area for messages that failed validation
Reconciliation	Compare totals across systems to prove nothing was lost or duplicated

Recommended decisions at a glance

First release	Batch MVP before Kafka and Spark
Financial truth	PostgreSQL orders, payments, and refunds
Behavior truth	Versioned Kafka events with stable event IDs
Stream guarantee	At-least-once delivery plus downstream deduplication
Raw format	Partitioned Parquet with ingestion metadata
Orchestration	Kestra; keep Bruin optional for later evaluation
Cloud	GCS and BigQuery created through Terraform
Portfolio data	Synthetic, deterministic, and free of real personal information

Your next practical step

Create the Phase 1 repository skeleton, define the PostgreSQL tables and seed generator, and write the exact revenue and order metric contracts before connecting any cloud service.